Ferramenta Baseada em Linguagem de Domínio Específico para o Apoio ao Ensino de Autômatos Finitos com Simulação Visual Interativa

João Thomaz Vieira Klaus Siegfried Beckmann

Bacharelado em Ciência da Computação
Universidade Tuiuti do Paraná
Orientador: Prof. Diógenes Cogo Furlan

Curitiba, 2026

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Projeto da DSL (Autosim)
5. Implementação
6. Demonstração e Resultados
7. Conclusão

Contexto e motivação

- O estudo de **autômatos finitos e linguagens formais** é um dos pilares da formação em Ciência da Computação (máquinas abstratas, limites da computação, compiladores).
- Apesar da importância, é uma disciplina **reconhecidamente desafiadora**.
- Segundo Morazán e Antunez (2014), a dificuldade vem de uma causa precisa:

  > *o estudante projeta autômatos e prova sua correção **sem poder experimentar** nem obter **retroalimentação imediata**, ao contrário do que ocorre na programação.*

- Há um distanciamento entre o **formalismo simbólico** e a **prática computacional**.

O problema: as ferramentas existentes

Ferramentas de apoio (como o **JFLAP**) adotam uma abordagem **exclusivamente visual** (arrastar e soltar). Isso traz limitações relevantes:

- A descrição do autômato **não é textual** $\rightarrow$ dificulta versionamento (Git) e compartilhamento.
- **Não há validação formal** da estrutura com mensagens de erro claras e localizadas.
- O estudante que erra na construção **não recebe um diagnóstico preciso** do problema.

Lacuna

Falta uma ferramenta que una o **rigor da descrição textual** ao **diagnóstico de um compilador** e à **interatividade da simulação visual**.

A proposta: Autosim

Uma ferramenta educacional que acopla diretamente:

DSL textual

Linguagem de domínio específico, declarativa e em português, para descrever AFDs e AFNs.

Interface gráfica interativa

Renderiza o diagrama a partir do texto e **anima a simulação passo a passo**.

Compilador pedagógico

Análises léxica, sintática e semântica, com **diagnósticos localizados** (linha e coluna).

Contribuição central

Acoplamento direto entre a descrição textual e a geração automática de uma interface gráfica sincronizada.

Objetivos

Objetivo geral: desenvolver uma ferramenta educacional que integra a descrição textual de autômatos, sua compilação pedagógica e sua simulação visual em um único ambiente.

Objetivos específicos

1. Projetar uma **DSL** textual para descrever AFDs e AFNs (gramática em EBNF);
2. Implementar um **compilador pedagógico** (léxico, sintático e semântico) com diagnósticos localizados;
3. Desenvolver um **simulador** de cadeias sobre os autômatos descritos;
4. Construir uma **interface gráfica** que renderize o diagrama e anime a simulação;
5. **Validar** a ferramenta com autômatos clássicos e cenários de erro induzido.

Autômatos finitos

Um **AFD** é uma quíntupla $M = (Q, \Sigma, \delta, q_0, F)$:

- Q — conjunto finito de estados;
- Σ — alfabeto de entrada;
- $\delta : Q \times \Sigma \to Q$ — função de transição;
- $q_0 \in Q$ — estado inicial;
- $F \subseteq Q$ — estados de aceitação.

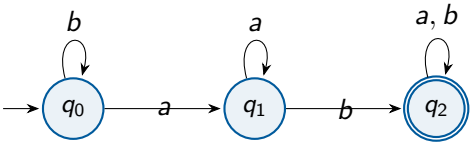
No **AFN**, a transição leva a um *conjunto* de estados ($\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$), admitindo transições vazias ε.

Linguagem reconhecida

$$L(M) = \{\, w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \,\}$$

AFD e AFN são **equivalentes** em poder de reconhecimento (construção de subconjuntos) e definem exatamente as **linguagens regulares**.

Autômatos finitos — um exemplo prático



Cada elemento de $M = (Q, \Sigma, \delta, q_0, F)$ no grafo:

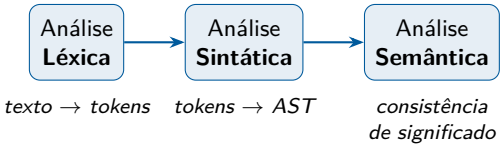
- $Q = \{q_0, q_1, q_2\}$ — os círculos;
- $\Sigma = \{a, b\}$ — os rótulos das arestas;
- q_0 — estado inicial (seta de entrada);
- $F = \{q_2\}$ — estados finais (círculo duplo);
- δ — as próprias arestas do grafo:

δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

Reconhece as cadeias sobre Σ que **contêm "*ab*" como subcadeia**. **Aceita:** `ab`, `aab`, `bab`. **Rejeita:** ε, `ba`, `bbb`.
Reaparece adiante — na DSL e na simulação.

Compiladores: as fases de análise

Um compilador traduz um programa-fonte verificando sua consistência em fases sucessivas:



- **Léxica:** agrupa caracteres em unidades léxicas (*tokens*) — via expressões regulares.
- **Sintática:** organiza os *tokens* em uma **Árvore Sintática Abstrata (AST)**.
- **Semântica:** verifica regras que a gramática não captura (referências, consistência).

Linguagens de Domínio Específico (DSL)

Definição (Fowler, 2010)

Uma DSL é "uma linguagem de programação de **expressividade limitada**, voltada para um **domínio específico**".

- Ganha **concisão e legibilidade** no seu domínio, ao custo da generalidade.
- **DSL interna:** embutida em uma linguagem hospedeira.
- **DSL externa:** sintaxe própria, exige um **compilador dedicado**.

*A Autosim é uma **DSL externa*** — com sintaxe própria e um compilador implementado em Rust.

Trabalhos relacionados e lacuna

JFLAP, GAM, Simulador UFSC e AutomatoGraph são **puramente visuais**:

Trabalho	DSL textual	Versionamento	Compilador/ diagnóstico	Simulação visual	Voltada ao ensino
Análise Léxica em Rust	Não	Sim	Não	Não	Não
JFLAP	Não	Não	Não	Sim	Sim
GAM	Não	Não	Não	Sim	Sim
Simulador UFSC	Não	Não	Não	Sim	Sim
AutomatoGraph	Não	Não	Não	Parcial	Sim
Autosim (este trabalho)	**Sim**	**Sim**	**Sim**	**Sim**	**Sim**

É essa lacuna que a Autosim ocupa.

A linguagem: um exemplo

DSL externa, sintaxe em **português**; cada arquivo `.asl` tem um alfabeto comum e um ou mais autômatos, seguidos de comandos de simulação.

```
alfabeto { 'a', 'b' }

automato AFD exemplo_afd {
    estados { q0, q1, q2 }
    inicial q0
    finais { q2 }
    transicoes {
        q0 -> q1 com 'a'      q1 -> q2 com 'b'
        q0 -> q0 com 'b'      q1 -> q1 com 'a'
        q2 -> q2 com 'a'      q2 -> q2 com 'b'
    }
}

simular exemplo_afd com "ab"      // aceita
simular exemplo_afd com "bbb"     // rejeita
```

Gramática formal (EBNF)

Linguagem **livre de contexto**, analisável por *parser* descendente preditivo com um símbolo de *lookahead*:

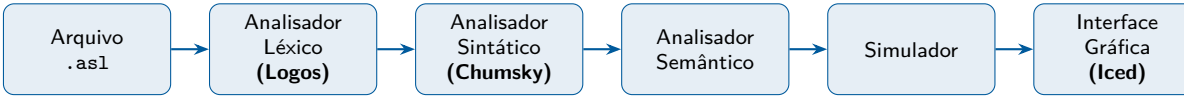
```
programa            ::= declaracao_alfabeto item*
declaracao_alfabeto ::= "alfabeto" "{" char_literal ("," char_literal)* [","] "}"
item                ::= declaracao_automato | comando_simulacao
declaracao_automato ::= "automato" tipo_automato ident "{"
                            "estados" "{" ident ("," ident)* [","] "}"
                            "inicial" ident
                            "finais"  "{" ident ("," ident)* [","] "}"
                            "transicoes" "{" transicao* "}" "}"
tipo_automato       ::= "AFD" | "AFN"
transicao           ::= ident "->" ident "com" (char_literal | "epsilon" | "eps")
comando_simulacao   ::= "simular" ident "com" cadeia_literal
```

11 palavras reservadas, *case-insensitive*.

Decisões de projeto

- **Declaração única de alfabeto** — reforça o conceito teórico e simplifica a verificação semântica (um único conjunto de referência).
- **Classificação explícita AFD/AFN** — permite verificações específicas por tipo (ex.: determinismo só para AFD).
- **Case-insensitive** — `AFD`, `afd`, `Afd` são equivalentes; reduz a carga cognitiva do estudante.
- **Suporte a transições** ε — palavras-chave `epsilon` / `eps` para AFNs com movimentos vazios.

Pipeline de compilação



- Fases executadas **sequencialmente**; em caso de erro, os diagnósticos são exibidos (via **Ariadne**) e a execução é interrompida antes da fase seguinte.
- Implementação em **Rust**, reaproveitando bibliotecas maduras do ecossistema.

Análise léxica e sintática

Léxica — Logos

- *Lexer* gerado a partir de anotações no tipo `Token`.
- Gera um AFD para reconhecimento eficiente dos padrões.
- Palavras reservadas *case-insensitive*; espaços e comentários (`//`) descartados.
- Erro léxico $\rightarrow$ posição (linha/coluna) do caractere inválido.

Sintática — Chumsky

- *Parser combinators*: a gramática é codificada compondo funções.
- Produz a **AST**, eliminando ruídos sintáticos (delimitadores).
- *Spans* referenciam índices de *token* para localizar erros.

Análise semântica — 3 fases estáticas

1. **Tabela de símbolos:** detecta símbolo/estado/autômato **duplicado**.
2. **Verificação de referências:** estado inicial/final/origem/destino não declarado; símbolo fora do alfabeto; simulação de autômato inexistente.
3. **Verificação de determinismo** (só para AFD):
   - ausência de transições ε;
   - unicidade de transição por par (estado, símbolo).

Os erros são **acumulados** e apresentados em conjunto — o estudante vê todos os problemas de uma só vez.

Simulador

- Executa a função de transição sobre a cadeia, registrando **todas as configurações intermediárias**.
- Exibe um **caminho único** do autômato, e não a nuvem de estados simultâneos:
  - cadeia aceita $\rightarrow$ o **caminho aceitador**;
  - cadeia rejeitada $\rightarrow$ o caminho que **mais se aproxima** da aceitação.
- Escolha por **busca em largura** sobre pares (estado, símbolos consumidos): maximiza os símbolos lidos e, no empate, minimiza a distância a um estado final.
- Guarda o **histórico completo** $\Rightarrow$ navegação **bidirecional** (avançar / retroceder) durante a execução.

Interface gráfica (Iced)

- **Renderização automática** do diagrama de estados a partir do texto-fonte.
- **Simulação passo a passo**, com reprodução automática (800 ms/passo): destaca o estado corrente, a transição percorrida e a cadeia consumida.
- **Navegação** do diagrama: *pan* (arrastar) e *zoom* (rolar), com botão **Centralizar**.
- Painel muda de cor conforme o **veredito**: **verde (aceita)** / **vermelho (rejeita)**.

Diagnóstico de erros (Ariadne)

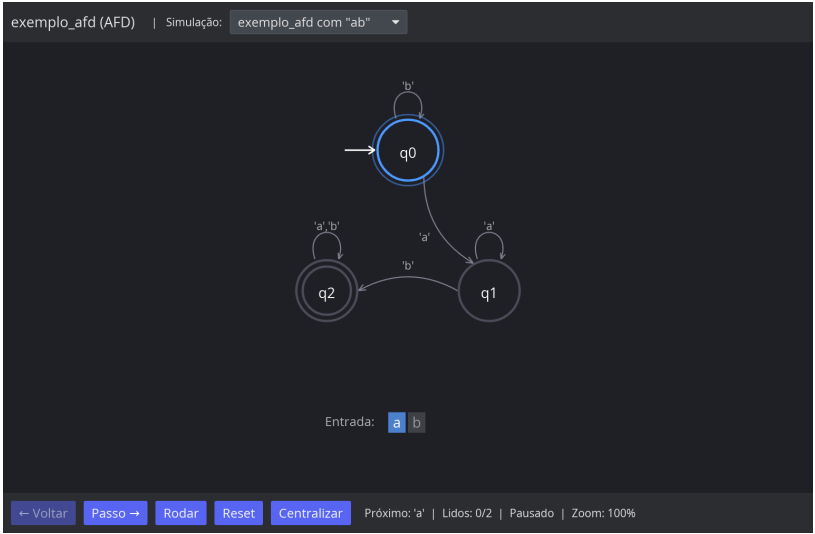
- Toda mensagem inclui **linha e coluna**, um **trecho do código com realce colorido** e uma **seta** apontando para o ponto do erro.
- Descrição do problema **em português**.

Exemplos

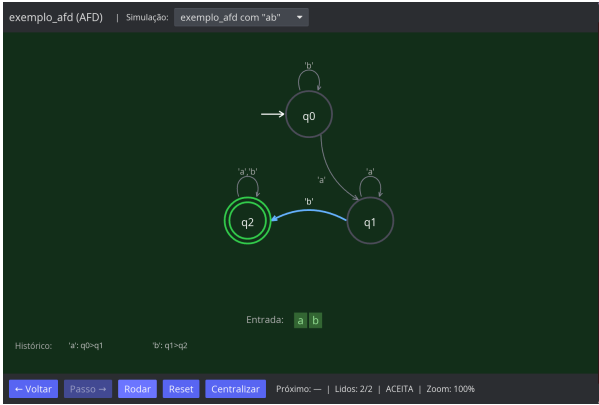
- Caractere `%` não reconhecido $\rightarrow$ *"token inválido"*.
- AFD com transição ε $\rightarrow$ *"AFD não pode ter epsilon"*.
- Falta a palavra `inicial` $\rightarrow$ *"esperava 'inicial' "*.

Substitui o "tentar e errar visual" por **retroalimentação imediata e precisa**.

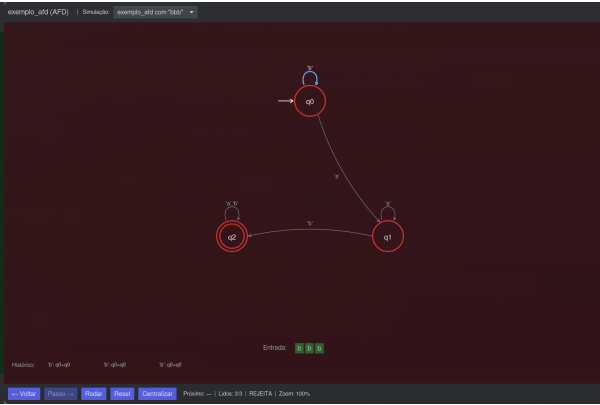
Interface — visão geral



Simulação — vereditos

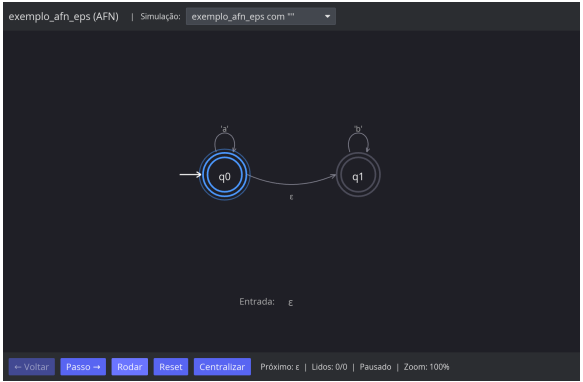


ACEITA — `"ab"`

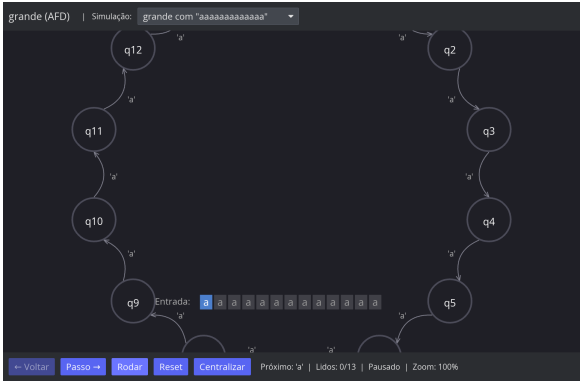


REJEITA — `"bbb"`

AFN-ε e navegação em diagramas grandes



AFN com transições ε



AFD de 14 estados — *pan* e *zoom*

Validação — testes e autômatos clássicos

51 testes automatizados

Analisador léxico	12
Analisador sintático	15
Analisador semântico	15
Simulador	9
Total	**51**

Autômatos de Hopcroft (2006) e Sipser (2012) — todos corretos:

Autômato	**Cadeia**	**Esperado**	**Obtido**
AFD: contém 01	1101	Aceita	Aceita
AFD: contém 01	111	Rejeita	Rejeita
AFN: termina em 01	001	Aceita	Aceita
AFN: termina em 01	010	Rejeita	Rejeita
AFN-ε: a^*b^*	aabb	Aceita	Aceita
AFN-ε: a^*b^*	ba	Rejeita	Rejeita
AFN-ε: a^*b^*	ε	Aceita	Aceita

Conclusão

Contribuições

- DSL em português (AFD/AFN-ε), formalizada em EBNF;
- Compilador pedagógico com **diagnósticos localizados**;
- Simulador com histórico bidirecional + GUI animada;
- Validação: **51 testes**, todos corretos.

Limites e futuro

- Alfabeto único; sem CLI textual nem editor embutido;
- Sem conversão AFN$\rightarrow$AFD / minimização;
- Futuro: editor com realce, conversões e estudo com estudantes.

Obrigado!

Perguntas?

João Thomaz Vieira · Klaus Siegfried Beckmann

Orientador: Prof. Diógenes Cogo Furlan · UTP, 2026